Manual para el curso Mineria de Datos y Análisis Multivariado (XS-0126)

Segundo examen

Autor: Anderson Martínez
Colaboradores: Sebastián Calvo, Justin Zhu, Fabián Almirall

Contents

1 Clasificación **2**
1.1 Apredizaje o entrenamiento . . . 2
1.2 Validación . . . 2
1.3 Predicción . . . 2
1.4 Regresión logística multinomial . . . 2
1.4.1 Supuesto . . . 3
1.4.2 Modelo . . . 3
1.4.3 Ejemplo aplicado en clase . . . 3
1.4.4 ¿Qué situaciones en la muestra nos podría generar problemas en la muestra? . . . 5
1.5 Validación cruzada . . . 5
1.6 Ejemplo con datos Iris . . . 6
1.6.1 Forma manual de muestreo balanceado . . . 7
1.6.2 Forma automática . . . 7
1.6.3 Gráfico para ver agrupaciones de forma descriptiva . . . 8
1.6.4 Modelo multi-logístico en r . . . 9
1.7 Validación del modelo de forma manual . . . 11
1.8 Validación del modelo de forma automática . . . 12

2 Aplicación de clasificación en caso de crédito **12**
2.1 ¿Cómo obtener el porcetaje de categorías en tablas de contigencia? . . . 13
2.1.1 Matrix de boxplot para analizar cuál variable (númerica) es mejor . . . 15
2.1.2 Tablas de contigencia para identificar variables categóricas correctas . . . 16
2.2 Creación de modelo multinomial . . . 17
2.3 Creación de Tabla de clasificación (Tabla de verdad) . . . 17
2.3.1 Indicadores de desempeño . . . 17
2.3.2 Matriz de Confusión: Detección de Tipo de Cliente . . . 18
2.4 Función eval para crear indicadores de desempeño . . . 19
2.5 Validación cruzada en r . . . 21
2.5.1 Validación cruzada con las direntes particiones y diferentes umbrales . . . 22

3 K-vecinos más cercanos **23**
3.0.1 Construcción de la matriz de distancias . . . 23
3.1 Función de k-vecinos, para diferentes tamaños de k . . . 26
3.2 Validación cruzada con k-vecinos . . . 27
3.2.1 Para un número k-vecinos en específico . . . 27
3.2.2 Para un diferentes números de k-vecinos . . . 28
3.2.3 Comparación de dos métodos de distancia una cierta cantidad de veces . . . 30

4 Métodos basados en árboles **35**
4.1 Validación cruzada, para el método de árboles 36
4.2 Gráfico de árboles 38

1 Clasificación

Clasificación consiste en asignar una observación con q variables en uno de varios grupos preestablecidos. El objetivo básico es construir una regla que tome la información de las q variables para resumirla en un indicador con el cual se pueda clasificar la observación de manera correcta en uno de los grupos. La regla se conoce como función discriminante. Varios métodos desarrollados para abordar el problema de clasificación:

- Análisis discriminante.
- Regresión logística (binomial/multinomial).
- K-vecinos más cercanos.
- Máquinas vectoriales de soporte.
- Métodos basados en árboles.

1.1 Apredizaje o entrenamiento

Se parte de un conjunto de observaciones cuya pertenencia a uno de los k grupos es conocida. Este conjunto se llama muestra de aprendizaje.

En cada grupo se tienen n_j observaciones medidas en q variables.

Con este conjunto de n observaciones ($n = \sum n_j$) se construye la función discriminante o regla de clasificación.

1.2 Validación

Se utiliza otro conjunto de datos para los cuales también se conoce su pertenencia a cada uno de los k grupos. Este conjunto se llama muestra de validación.

Se aplica la función discriminante a ese conjunto de datos y se evalúa qué tan bien fueron clasificados.

La ventaja de hacer esto es que estos individuos no participaron en la construcción de la regla.

Sin embargo, si hay pocos datos se puede usar como muestra de validación la misma muestra de entrenamiento.

Existen métodos secuenciales de validación.

1.3 Predicción

Una vez que la regla ha sido validada se puede aplicar a nuevos datos para los cuales no se conoce su pertenencia a uno de los k grupos.

Se confía en que la regla hará una buena clasificación.

1.4 Regresión logística multinomial

Anteriormente en el curso de modelos avanzados vimos que la regresión logística era:

$$log\left(\frac{\pi_x}{1-\pi_x}\right) = \beta_0 + \beta_1 X_1 + ... + \beta_n X_n$$

donde teníamos el siguiente caso:

$$Y < \begin{matrix} 0 \\ 1 \end{matrix} \implies \pi_x = P(Y = 1 \mid X)$$

Por lo tante la regresión logística multinomial es

Es una extensión del modelo de regresión logístico cuando la respuesta condicional es categórica pero hay más de dos categorías.

Entra dentro de los modelos lineales generalizados.

Puede incluir tanto variables métricas como categóricas en la parte sistemática, es decir, en las variables independientes.

Las variables métricas no deben seguir ninguna distribución particular. (LAS X'S NO SON NORMALES)

Matemáticamente se veería de la siguiente manera:

$$Y \left\langle \begin{array}{lll} a & \implies & \pi_a(x) = P(Y = a \mid X) \\ b & \implies & \pi_b(x) = P(Y = b \mid X) \\ c & \implies & \pi_c(x) = P(Y = c \mid X) \\ d & \implies & \pi_d(x) = P(Y = d \mid X) \end{array} \right.$$

1.4.1 Supuesto

Se basa en el supuesto de independencia de alternativas irrelevantes (IIA, siglas en inglés), lo que no siempre es deseable.

Las probabilidades de preferir una clase sobre otra no dependen de la presencia o ausencia de otras alternativas "irrelevantes".

Por ejemplo, las probabilidades relativas de ir al trabajo en automóvil o autobús no cambian si se agrega una bicicleta como posibilidad adicional.

Los individuos a menudo violan esta suposición al tomar decisiones.

Si las opciones incluyen un automóvil y un autobús azul y se introduce la opción de un autobús rojo, una persona puede ser indiferente entre un autobús rojo y un autobús azul. Aquí, la opción del autobús rojo no es irrelevante, porque un autobús rojo es un sustituto perfecto del autobús azul.

1.4.2 Modelo

El modelo se utiliza para calcular la probabilidad de que el i-ésimo individuo pertenerza al j-ésimo grupo, tomando como referencia el grupo 1 (puede tomarse cualquier otro grupo).

Se deriva un conjunto de $k - 1$ ecuaciones.

Cada ecuación tiene un vector de coeficientes $\beta^{(j)}$.

$$log\left(\frac{\pi_j}{\pi_1}\right) = \beta_0^{(j)} + \beta_1^{(j)} X_1 + ... + \beta_q^{(j)} X_q$$

1.4.3 Ejemplo aplicado en clase

Para este ejemplo nuestra variable respuesta será de que depende el medio de transporte según la cantidad de kilométros.

Hay tres respuestas:

1: Transporte Público

2: Moto o automóvil

3: Bicicleta o caminando

Y	D
1	4
1	12
1	21
2	11
2	12
2	1
2	2
2	5
2	9

Lo que trata el modelo es crear un modelo para predecir nuevas unidades de forma certera, un ejemplo de aplicación de esto puede ser en fraude o en casas de apuestas.

Note que en el modelo se pone la primera categoría de referencia en este caso es π_1.

Siguiendo con el mismo ejemplo como tenemos 3 categorías tenemos dos ecuaciones del modelo

$$log\left(\frac{\pi_2}{\pi_1}\right) = \beta_0^{(2)} + \beta_1^{(2)} D$$

$$log\left(\frac{\pi_3}{\pi_1}\right) = \beta_0^{(3)} + \beta_1^{(3)} D$$

La regla general de cuántos coeficientes tiene que calcular el modelo es $(k-1)\cdot p$, donde k es el número de categorías y p el número de coeficientes por ecuación.

Riego relativo

De estas ecuacione podemos obtener el riesgo relativo de la siguiente manera:

$$\frac{\pi_2}{\pi_1}$$

Teniendo en cuenta el ejemplo anterior y la fórmula del riesgo relativo, tenemos lo siguiente:

$$\frac{\pi_2}{\pi_1} = e^{\beta_0^{(2)} + \beta_1^{(2)} D} = A \qquad \frac{\pi_3}{\pi_1} = e^{\beta_0^{(3)} + \beta_1^{(3)} D} = B$$

Para simplificar el proceso asuma que los RR son A y B

Despejamos π_1 en las dos ecuaciones

$$\pi_1 = \frac{\pi_2}{A} \qquad \pi_1 = \frac{\pi_3}{B}$$

Igualamos las dos ecuaciones de π_1

$$\frac{\pi_2}{A} = \frac{\pi_3}{B}$$

De esta ecuación despejamos π_2

$$\pi_2 = \frac{A\pi_3}{B}$$

Sabemos que:

$$\pi_1 + \pi_2 + \pi_3 = 1$$

Reemplazamos π_2 y nos lleva al siguiente resultado

$$\pi_1 + \frac{A\pi_3}{B} + \pi_3 = 1$$

Despejando los tres π llegamos al siguiente resultado:

$$\pi_3 = \frac{B}{1+A+B} \qquad \pi_2 = \frac{A}{1+A+B} \qquad \pi_1 = \frac{1}{1+A+B} \text{ ó } 1 - \left(\frac{B}{1+A+B} + \frac{A}{1+A+B}\right)$$

1.4.4 ¿Qué situaciones en la muestra nos podría generar problemas en la muestra?

Si tenemos el siguiente caso donde un grupo tiene 0 unidades de estudio es un gran problema ya que nos genera problemas al momento de hacer el modelo, por lo menos para solucionar el problema deber haber 1 unidad. En ocasiones no se realizan estas tablas de contingencia y se generan problemas cuando se hace el modelo.

	Y = 1	Y = 2	Y = 3
H	30	60	0
M	30	10	20

Otro ejemplo es el siguiente

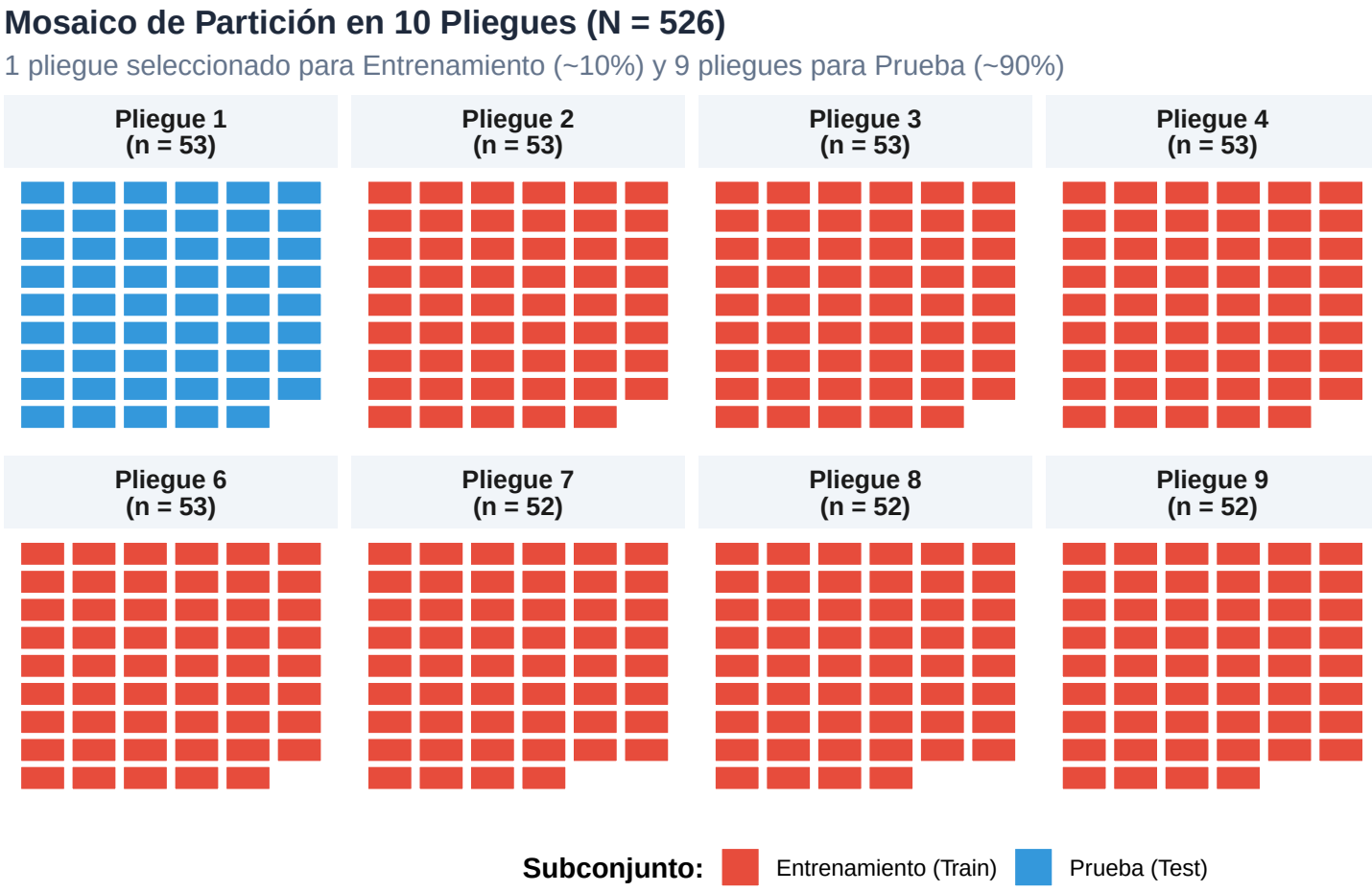
Si alguna de las categorías de respuesta nos da como un 1% o hasta 5% y las otras son muy grandes nos puede dar problemas ya que no está del todo balanceado para generar el modelo. Esto se nota mucho cuando son por ejemplo 1000 datos y una categoría solo tiene un 1% corresponderá a 10 unidades mientras que las otras tienen 400 lo cuál sería muy desbalanceado.

	Y = 1	Y = 2	Y = 3
H	30	60	1
M	30	10	20
Total	60	70	21
(%) total	40%	46%	14%

1.5 Validación cruzada

Suponga que tiene una base de datos de $n = 526$

A continuación se le detallan los pasos de este proceso



1. Ajustar un modelo de 9 pliegues (fit)
2. Uso el modelo para estimar valores de la base de datos del test.
3. Comparar el valor estimado con el valor observado, esto es una medida de ajuste $y - \hat{y}$, pero como en este caso la respuesta es categoríca el resultado se debe observar como la siguiente tabla:

Observado	Estimado	1	2	3	Total Observado
	1	15	45	35	95
	2	75	10	90	175
	3	50	19	18	87
	Total Estimado	140	74	143	357

Para un total Total de casos evaluados: $n = 357$

Clasificaciones correctas (diagonal principal): $15 + 10 + 18 = 43$. Lo ideal es que este número sea más alto porque para este caso la tasa de exactiud fue de Accuracy $= \frac{43}{357} \approx 12{,}04\%$

1.6 Ejemplo con datos Iris

```
  load("iris.Rdata")
  RNGkind(sample.kind = "Rounding") #Esto es para estandarizar la semilla según la versión de R
  set.seed(10)
  n=nrow(base)
  m     = sample(1:n,100)
  basea = base[m,] # Selecciona los números de la muestra creada en m
  basev = base[-m,] # Elimina los número seleccionados en m
```

```
table(basea$especie)
```

```
##
##     setosa versicolor  virginica
##         34         34         32
```

Si se diera el caso de que se quiere la misma cantidad de unidades por especie existen dos formas para realizar este tipo de muestreo:

1.6.1 Forma manual de muestreo balanceado

```
basese = base[base$especie == "setosa",]
n=nrow(basese)
  mse     = sample(1:n,33)
  basease = basese[mse,]
  basevse = basese[-mse,]
```

```
basevir = base[base$especie == "virginica",]
n=nrow(basevir)
  mvir    = sample(1:n,33)
  baseavir = basevir[mvir,]
  basevvir = basevir[-mvir,]
```

```
basever = base[base$especie == "versicolor",]
n=nrow(basever)
  mver     = sample(1:n,33)
  baseaver = basever[mver,]
  basevver = basever[-mver,]
```

```
base_comple= rbind(basease,baseavir,baseaver)
base_rest =  rbind(basevse,basevvir,basevver)
```

```
table(base_comple$especie) #muestra por especie
```

```
##
##     setosa versicolor  virginica
##         33         33         33
```

```
table(base_rest$especie) # Resto de cada una de las especies
```

```
##
##     setosa versicolor  virginica
##         17         17         17
```

1.6.2 Forma automática

```
library(dplyr)
muestra = base %>%
```

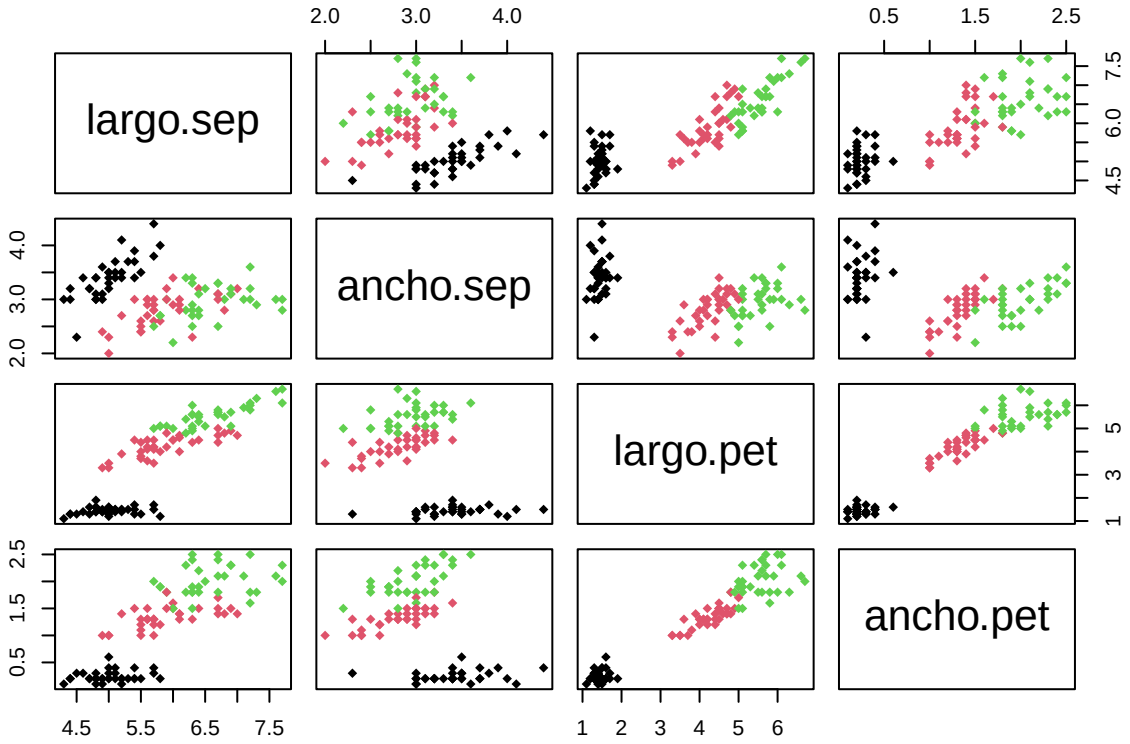
```
  group_by(especie) %>%
  slice_sample(n = 33, replace = FALSE) %>%
  ungroup()
```

```
table(muestra$especie)
```

```
##
##     setosa versicolor  virginica
##         33         33         33
```

1.6.3 Gráfico para ver agrupaciones de forma descriptiva

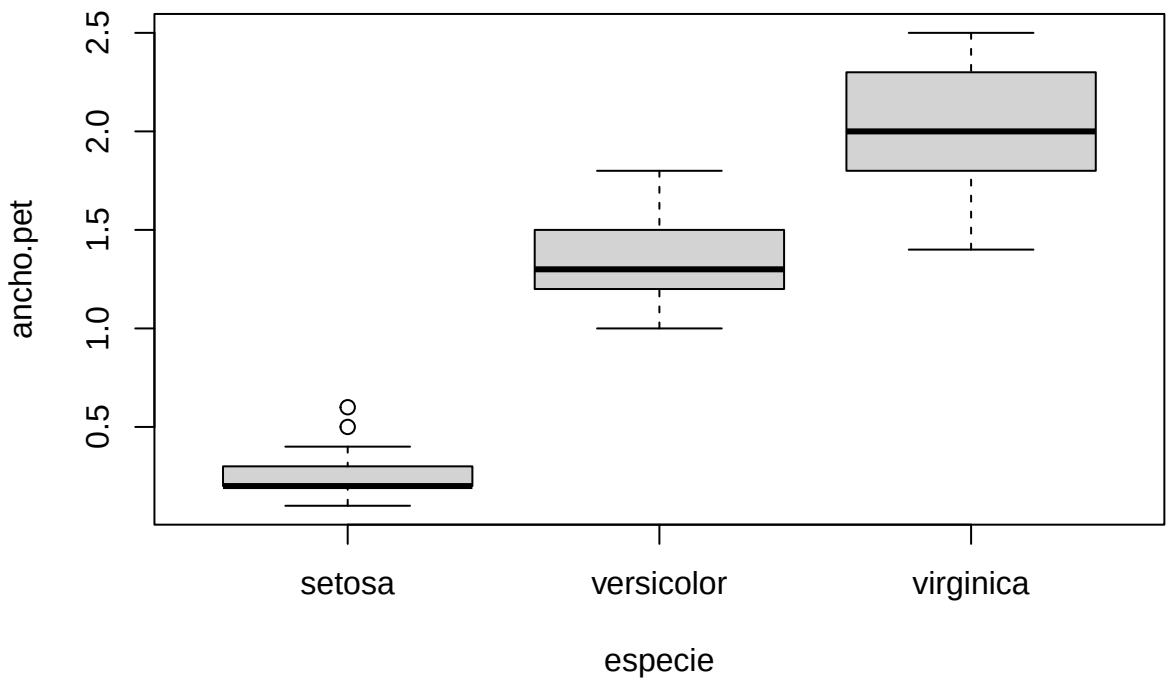
```
sp=unclass(basea$especie)
pairs(basea[,-5],col=sp,pch=18)
```



Cuándo hay separaciones perfectas por ejemplo con la variable `ancho.pet` el modelo puede identificar perfectamente las clases, lo que causas que lso coeficientes de regresión logística dirverjan hacia el infinito y no se pueden obtener estimaciones estables.

Esto también lo podemos verificar por medio de un `boxplot()`, para que se observe bien el gráfico se tiene que colocar de forma que `boxplot(X~Y, data = base)`

```r
boxplot(ancho.pet~especie, data = base)
```



En este caso se traslapan un poco por lo que el modelo podría funcionar. Sin embargo, si están separados totalmente es lo que no puede generar problemas con la convergencia de los coeficientes.

1.6.4 Modelo multi-logístico en r

```r
library(nnet)
mod1=multinom(especie~.,basea)
```

```
## # weights:  18 (10 variable)
## initial  value 109.861229
## iter  10 value 9.274654
## iter  20 value 0.810363
## iter  30 value 0.140935
## iter  40 value 0.044246
## iter  50 value 0.041235
## iter  60 value 0.034833
## iter  70 value 0.024835
## iter  80 value 0.020949
## iter  90 value 0.015308
## iter 100 value 0.013757
## final  value 0.013757
## stopped after 100 iterations
```

```
mod1$convergence
```

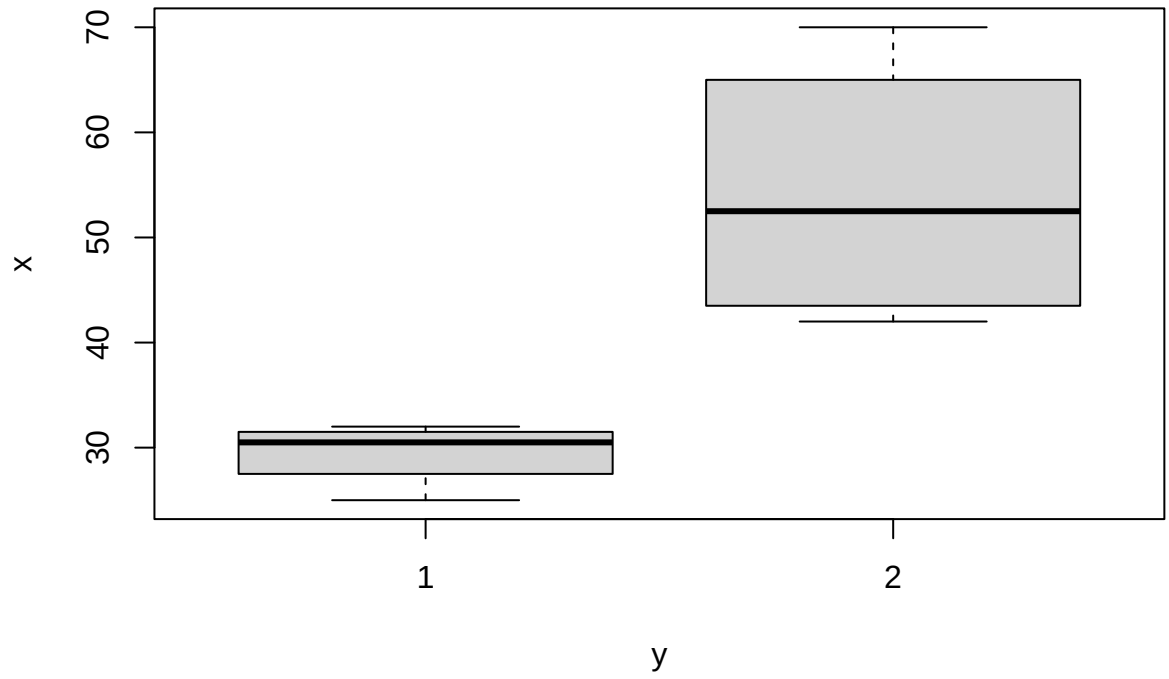
```
## [1] 1
```

En este caso podemos ver que el modelo si convergió

A continuación observe el siguiente caso

```
x = c(25,30,32,31,42,45,60,70)
y = c(1,1,1,1,2,2,2,2)
y = factor(y)
```

```
boxplot(x~y)
```




```
modx=multinom(y~x)
```

```
## # weights:  3 (2 variable)
## initial  value 5.545177
## iter  10 value 0.688621
## iter  20 value 0.170540
## iter  30 value 0.131731
## iter  40 value 0.106057
## iter  50 value 0.055059
## iter  60 value 0.050782
## iter  70 value 0.043708
## iter  80 value 0.038152
## iter  90 value 0.012647
## iter 100 value 0.006450
```

```
## final  value 0.006450
## stopped after 100 iterations
```

```
modx$convergence
```

```
## [1] 1
```

En este casi el modelo si convergió pero observe los errores estándar

```
summary(modx)
```

```
## Call:
## multinom(formula = y ~ x)
##
## Coefficients:
##                  Values Std. Err.
## (Intercept) -43.880846 87.648885
## x             1.184744  2.390251
##
## Residual Deviance: 0.01290036
## AIC: 4.0129
```

Observe que el error estándar es 87.64 lo cuál es extremandamente grande.Por lo que la estimación del coeficiente no es para nada confiable. Por lo que se afecta la credibilidad de la estimación.

1.7 Validación del modelo de forma manual

Observe el individuo número 32 de la base de validación

```
basev[32,]
```

```
##    largo.sep ancho.sep largo.pet ancho.pet    especie
## 99       5.1       2.5         3       1.1 versicolor
```

Observe la posición de los coeficientes

```
summary(mod1)
```

```
## Call:
## multinom(formula = especie ~ ., data = basea)
##
## Coefficients:
##            (Intercept)  largo.sep  ancho.sep largo.pet ancho.pet
## versicolor    13.85687  -9.317049  -3.369218  19.13456 -9.822023
## virginica    -28.16696 -25.741794 -50.511522  62.18905 33.038330
##
## Std. Errors:
##            (Intercept) largo.sep ancho.sep largo.pet ancho.pet
## versicolor    1568.757  525.8587  279.8332  647.5993  614.7029
## virginica     1623.508  527.4245  302.0196  653.4406  624.5678
##
## Residual Deviance: 0.0275148
## AIC: 20.02751
```

```
x=cbind(1,basev[32,1:4])
beta1=summary(mod1)$coefficients[1,] # Versicolor
beta2=summary(mod1)$coefficients[2,] #virginica
```

```
PL1=sum(x*beta1)
PL2=sum(x*beta2)
```

Probabilidad de pertenecer al grupo de Versicolor

```
den=1+exp(PL1)+exp(PL2)
round(exp(PL1)/den,3)
```

```
## [1] 0.989
```

Probabilidad de pertecer al grupo de Virginica

```
round(exp(PL2)/den,3)
```

```
## [1] 0
```

Probabiliad de setosa

```
round(1/den,3)
```

```
## [1] 0.011
```

o la otra forma

```
round(1-(exp(PL1)/den+exp(PL2)/den),3)
```

```
## [1] 0.011
```

1.8 Validación del modelo de forma automática

```
probs=predict(mod1,basev,type="probs")
round(probs[32,],3)
```

```
##     setosa versicolor  virginica
##      0.011      0.989      0.000
```

Tabla de confusión

```
sp2=predict(mod1,basev)
  table(basev$especie,sp2)
```

```
##             sp2
##              setosa versicolor virginica
##   setosa         16          0         0
##   versicolor      0         13         3
##   virginica       0          0        18
```

2 Aplicación de clasificación en caso de crédito

```
load("Credit.Rdata")
base2$sobreendeudado = factor(base2$sobreendeudado)
```

```
table(base2$cliente,base2$sobreendeudado)
```

```
##
##            0    1
##   bueno 3075  121
##   malo  1096  153
```

El 0 representa que no está sobre endudado y el 1 que si lo está, en la tabla anterior se puede ver como hay más clientes malos que están sobreendeudados

2.1 ¿Cómo obtener el porcetaje de categorías en tablas de contigencia?

Ejemplo 1 ¿que porcentaje de los NO sobreenduedados son buenos?

```
prop.table(table(base2$cliente,base2$sobreendeudado),2)*100 # el 2 significa que lo saca por filas es d
```

```
##
##                0        1
##   bueno 73.72333 44.16058
##   malo  26.27667 55.83942
```

Vea que en esete caso el 74% de los no sobreenduados son buenos clientes y el 26 de los no sobreendeudados son malos clientes.*Note que las sumas de las columnas dan 100*

Ejemplo 1 ¿que porcentaje de los buenos clientes son sobreendeudados?

```
prop.table(table(base2$cliente,base2$sobreendeudado),1)*100
```

```
##
##                 0         1
##   bueno 96.214018  3.785982
##   malo  87.750200 12.249800
```

Casi que el 4% de los clientes buenos están sobrendeudados. *Note que la suma de las filas da 100%*

Si pasara el caso de que tenemos que el 45% de los sobreenduedados son buenos clientes y el 48% de los NO sobreendudados son buenos clientes , los porcentajes son tan similares que el modelo no podrá clasificar, para que el modelo funcione siempre tiene que existir un tipo de discriminación.

Se hace el mismo ejericio, pero con otras variables

```
prop.table(table(base2$cliente,base2$estado_civil),2)*100
```

```
##
##          soltero   casado separado divorciado    viudo
##   bueno 66.49538 74.48255 50.76923   63.15789 71.64179
##   malo  33.50462 25.51745 49.23077   36.84211 28.35821
```

El porcentaje de clientes buenos que son casados es de 75%

```
prop.table(table(base2$cliente,base2$vivienda),2)*100
```

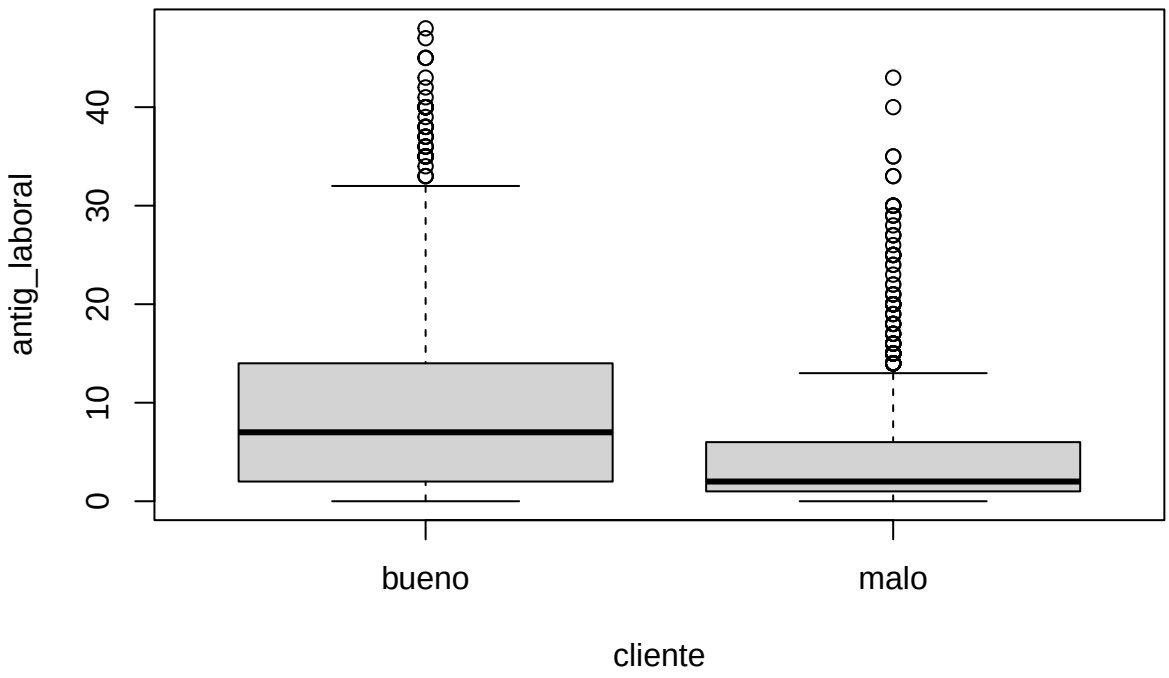
```
##
##           propia   padres alquilada contrato privado    otros
##   bueno 81.47268 70.33248  60.02014         65.85366 54.23197
##   malo  18.52732 29.66752  39.97986         34.14634 45.76803
```

El porcentaje más alto de buenos clientes lo tienen los clientes que poseen casa propia con un 81% los clientes que tienen casa propia que son buenos.

Nota importante: Si en una categoría hay muuy pocas observaciones puede ser peligroso porque al momento de partir la base de datos pueden exisitir partes de las base que no tengan eninguna observación en esa categoría, debido a que como es aleatorio y la cantidad de obervaciones es poca.

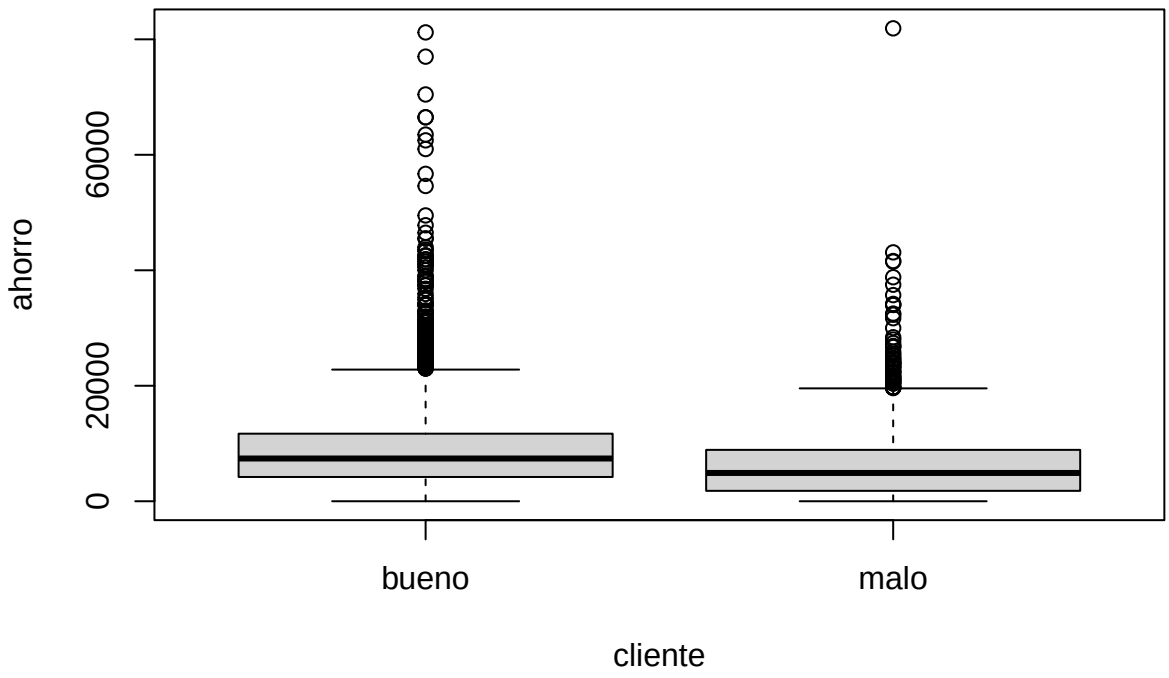
Las variables que son continuas como `antig_laboral`, `ahorro`, lo representando por medio de un boxplot

```
boxplot(antig_laboral~cliente, data = base2 )
```



Normalmente una persona con 0 años de antigüedad laboral uno diría que tiende a ser mal cliente, pero por esta razón es que la regresión multinomial toma en cuenta distintos facotres, como por ejemplo si está casado y tiene casa propia.

```
boxplot(ahorro~cliente, data = base2 )
```



Para esta situación cuando haya traslape es un indicador que el análisis va por buen camino, lo que no estaría bien es que las dos cajas tuverian el mismo tamaño

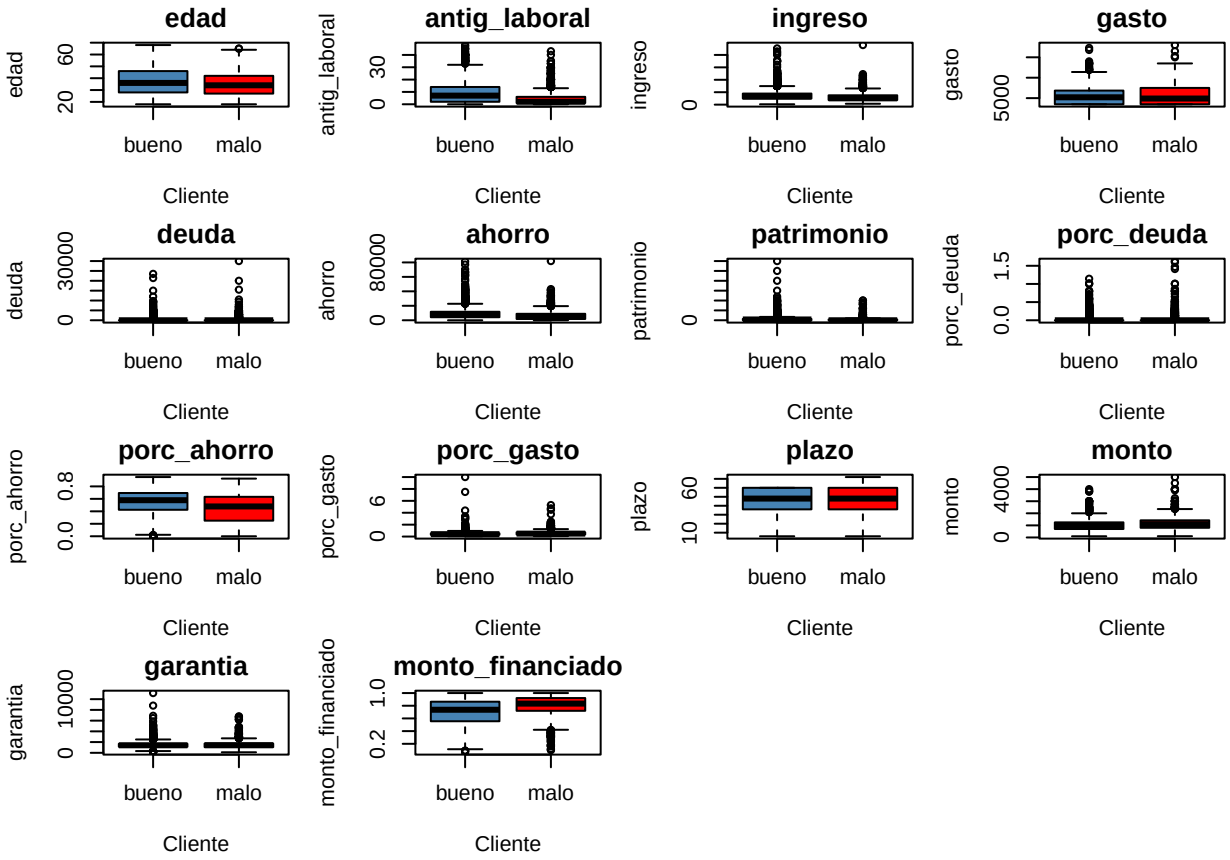
2.1.1 Matrix de boxplot para analizar cuál variable (númerica) es mejor

```
# Configurar la ventana gráfica (ej. 4 filas x 4 columnas)
par(mfrow = c(4, 4), mar = c(4, 4, 2, 1))

vars_num <- c("edad", "antig_laboral", "ingreso", "gasto", "deuda",
              "ahorro", "patrimonio", "porc_deuda", "porc_ahorro",
              "porc_gasto", "plazo", "monto", "garantia", "monto_financiado")

for (var in vars_num) {
  boxplot(base2[[var]] ~ base2$cliente,
          main = var,
          xlab = "Cliente",
          ylab = var,
          col = c("steelblue", "red"))
}

# Restaurar parámetros gráficos por defecto
par(mfrow = c(1, 1))
```



2.1.2 Tablas de contigencia para identificar variables categóricas correctas

```
# Variables categóricas a iterar
vars_cat <- c("vivienda", "estado_civil", "trabajo", "sobreendeudado")

for (var in vars_cat) {
  cat("\n--- Proporciones de Cliente vs", toupper(var), "---\n")

  # Tabla cruzada dinámica manteniendo 'cliente' en las filas y margin = 1
  tabla_pct <- prop.table(table(base2$cliente, base2[[var]]), 2) * 100

  # Se imprime redondeando a 2 decimales para mayor claridad
  print(round(tabla_pct, 2))
}
```

```
##
## --- Proporciones de Cliente vs VIVIENDA ---
##
##         propia padres alquilada contrato privado otros
##   bueno  81.47  70.33     60.02            65.85 54.23
##   malo   18.53  29.67     39.98            34.15 45.77
##
## --- Proporciones de Cliente vs ESTADO_CIVIL ---
##
##         soltero casado separado divorciado viudo
##   bueno   66.50  74.48    50.77      63.16 71.64
```

```
##   malo    33.50  25.52    49.23      36.84 28.36
##
## --- Proporciones de Cliente vs TRABAJO ---
##
##         asalariado independiente temporal otros
##   bueno      79.31         67.55    40.13 60.23
##   malo       20.69         32.45    59.87 39.77
##
## --- Proporciones de Cliente vs SOBREENDEUDADO ---
##
##             0     1
##   bueno 73.72 44.16
##   malo  26.28 55.84
```

2.2 Creación de modelo multinomial

```
modx = glm(cliente ~ ., family = binomial, data = base2)
```

```
contrasts(base2$cliente)
```

```
##       malo
## bueno    0
## malo     1
```

Observe que en este caso bueno está codificado con un 0 y malo con un 1. Por lo que el modelo calcula la probabilidad de que sea malo y la probabilidad de que sea bueno la podemos obtener mediante el complemento.

```
pred=predict(modx,base2, type = "response") #sino se coloca el response nos da la parte lineal del mode
pred= ifelse(pred >= 0.2,1,0) # otra forma para colocarlo en un valor en específico de clasificación
pred= factor(pred)
levels(pred) = c("Buenos", "Malos")
```

```
table(pred)
```

```
## pred
## Buenos  Malos
##   2049   2396
```

2.3 Creación de Tabla de clasificación (Tabla de verdad)

	Valor Predicho	
Valor Real	Negativo	Positivo
Negativo	a	b
Positivo	c	d

2.3.1 Indicadores de desempeño

Error (e): es la proporción del número total de predicciones que son incorrectas respecto al total.
Nota: este indicador es a veces engañoso y debe ser siempre analizado en relación al tamaño de las clases.

$$e = \frac{b+c}{a+b+c+d}$$

Falsos Positivos (FP): es la proporción de casos negativos que fueron clasificados incorrectamente como positivos.

$$FP = \frac{b}{a+b}$$

Falsos Negativos (FN):es la proporción de casos positivos que fueron clasificados incorrectamente como negativos.

$$FN = \frac{c}{c+d}$$

Precisión (P): es la proporción del número total de predicciones que son correctas respecto al total.
Nota: este indicador es a veces engañoso y debe ser siempre analizado en relación al tamaño de las clases.

$$P = \frac{a+d}{a+b+c+d}$$

Precisión Positiva (PP): es la proporción de los casos positivos que fueron predichos correctamente.

$$PP = \frac{d}{c+d}$$

Precisión Negativa (PN): es la proporción de los casos negativos que fueron predichos correctamente.

$$PN = \frac{a}{a+b}$$

Acertividad Positiva (AP): indica la proporción de buena predicción para los positivos.

$$AP = \frac{d}{b+d}$$

Acertividad Negativa (AN):indica la proporción de buena predicción para los negativos.

$$AN = \frac{a}{a+c}$$

También hay estas equivalencias: $FP = 1 - PN$ y $PP = 1 - FN$

2.3.2 Matriz de Confusión: Detección de Tipo de Cliente

Real Predicho	Predicho: Mal cliente ($\hat{Y} = 1$)	Predicho: Buen cliente ($\hat{Y} = 0$)
Real: Mal cliente ($Y = 1$)	**Verdadero Positivo (VP)**	**Falso Negativo (FN)**
Real: Buen cliente ($Y = 0$)	**Falso Positivo (FP)**	**Verdadero Negativo (VN)**

Se comparan los valores predichos con los observados

```
table(base2$cliente,pred)# colocar primero los observados y luego los predichos
```

```
##         pred
##          Buenos Malos
##   bueno   1867  1329
##   malo     182  1067
```

Con base a esos resultados llegamos a la siguiente tabla

Real Predicho	Predicho: Bueno ($\hat{Y}=0$)	Predicho: Malo ($\hat{Y}=1$)	Total Real
Bueno ($Y=0$)	**1867** *(VN)*	**1329** *(FP)*	**3196**
Malo ($Y=1$)	**182** *(FN)*	**1067** *(VP)*	**1249**
Total Predicho	**2049**	**2396**	**4445**

Calculamos el porcentaje de cada categoría de predicción

```
prop.table(table(base2$cliente,pred),1)
```

```
##         pred
##             Buenos     Malos
##   bueno 0.5841677 0.4158323
##   malo  0.1457166 0.8542834
```

Real Predicho	Predicho: Bueno ($\hat{Y}=0$)	Predicho: Malo ($\hat{Y}=1$)	Total Real
Bueno ($Y=0$)	**42,00%** *(VN)*	**29,90%** *(FP)*	**71,90%**
Malo ($Y=1$)	**4,10%** *(FN)*	**24,00%** *(VP)*	**28,10%**
Total Predicho	**46,10%**	**53,90%**	**100,00%**

El porcentaje más peligroso corresponde al 4,10% de Falsos Negativos (clientes catalogados erróneamente como buenos siendo en realidad malos, que representan un 14,57% de toda la morosidad real). En riesgo financiero y colocación crediticia, este error implica desembolsar capital líquido a sujetos que terminarán en impago (default), lo cual genera pérdidas patrimoniales directas e irrecuperables sobre el balance de la institución. En contraste, el 29,90% de falsos positivos solo representa un costo de oportunidad (intereses no percibidos por rechazar a un cliente solvente), haciendo que el falso negativo sea, por asimetría de costos, la falla crítica más severa para la solvencia de la cartera.

2.4 Función eval para crear indicadores de desempeño

```
eval = function(y, pred) {
  confu = table(y, pred)
  e = 1 - sum(diag(confu)) / sum(confu)
  falsos = 1 - diag(confu) / apply(confu, 1, sum)
  error = c(e, falsos) * 100
  names(error) = c("e", "FP", "FN")
  return(list(Matriz = confu, Error = error))
}
```

Ejemplo de uso

```
mod = glm(cliente~edad+antig_laboral+vivienda, family = "binomial", data = base2) #Primero se debe tene
pred = factor(predict(mod, type = "response") > 0.5)
levels(pred) = c("bueno", "malo") # en este caso 0 es bueno y 1 es malo
(tab1 = table(base2$cliente,pred))
```

```
##        pred
##         bueno malo
##   bueno  3012  184
##   malo    974  275
```

Luego se puede sacar el porcentaje de Falsos Postivos y Falsos Negativos con `prop.table()` en las filas

```
(pt1 = prop.table(tab1,1))
```

```
##        pred
##              bueno       malo
##   bueno 0.94242804 0.05757196
##   malo  0.77982386 0.22017614
```

El error también lo podemos obtener de la siguiente manera:

```
(pt2 = prop.table(tab1))
```

```
##        pred
##              bueno       malo
##   bueno 0.67761530 0.04139483
##   malo  0.21912261 0.06186727
```

```
FP = pt1[1,2]
FN = pt1[2,1]
e = pt2[1,2] +pt2[2,1]
c(e,FP,FN)*100
```

```
## [1] 26.051744  5.757196 77.982386
```

```
(e1 = eval(base2$cliente, pred))
```

```
## $Matriz
##        pred
## y       bueno malo
##   bueno  3012  184
##   malo    974  275
##
## $Error
##         e        FP        FN
## 26.051744  5.757196 77.982386
```

Observe que lso resultados dan igual con la función y el método manual.

```
load("iris.Rdata")
```

```
library(nnet)
mod1= multinom(especie~., data = base)
```

```
## # weights:  18 (10 variable)
## initial  value 164.791843
## iter  10 value 16.177348
## iter  20 value 7.111438
## iter  30 value 6.182999
## iter  40 value 5.984028
## iter  50 value 5.961278
## iter  60 value 5.954900
## iter  70 value 5.951851
## iter  80 value 5.950343
```

```
## iter  90 value 5.949904
## iter 100 value 5.949867
## final  value 5.949867
## stopped after 100 iterations
```

```
pred = predict(mod1)
y = base$especie
tab = table(y,pred)
```

```
tab
```

```
##             pred
## y            setosa versicolor virginica
##   setosa         50          0         0
##   versicolor      0         49         1
##   virginica       0          1        49
```

En este caso los que están clasificados como setosa son los de la primera columna, en este caso no fue ninguna. Para la especie Versicolor los clasificados incorrectamente se ve en la segunda columna, en donde para esta situación hay 1 que es en realidad virginica. Por último, para la especie Virginica se clasificó uno mal como versicolor que es el mismo que mencionamos antes.

```
FS = sum(tab[2:3,1]) # Falsos setosa
Fver = sum(tab[c(1,3),2]) # Falsos Versicolor
Fvir = sum(tab[1:2,3]) #Falsos Virginica
```

2.5 Validación cruzada en r

```
library(caret)
part = createFolds(1:nrow(base2), k = 10) # desde 1 hasta n filas, partido en 10 partes distintas
class(part) #vea que es de objeto lista
```

```
## [1] "list"
```

```
summary(part)
```

```
##        Length Class  Mode
## Fold01 445    -none- numeric
## Fold02 444    -none- numeric
## Fold03 445    -none- numeric
## Fold04 444    -none- numeric
## Fold05 445    -none- numeric
## Fold06 444    -none- numeric
## Fold07 444    -none- numeric
## Fold08 444    -none- numeric
## Fold09 445    -none- numeric
## Fold10 445    -none- numeric
```

```
test = base2[part[[1]],] #obtiene las filas de la primera partición
train = base2[-part[[1]],]
```

```
c(nrow(test), nrow(train))
```

```
## [1]  445 4000
```

Hacemos el modelo

```r
mod = glm(cliente~edad+antig_laboral+vivienda, family = "binomial", data = train) #Primero se debe tene
pred = factor(predict(mod,test,type = "response") > 0.5)
levels(pred) = c("bueno", "malo") # en este caso 0 es bueno y 1 es malo
```

```r
(e2 = eval(test$cliente, pred))
```

```
## $Matriz
##        pred
## y       bueno malo
##   bueno   297   17
##   malo     95   36
##
## $Error
##         e        FP        FN
## 25.168539  5.414013 72.519084
```

2.5.1 Validación cruzada con las direntes particiones y diferentes umbrales

```r
# Se crea la función de evaluar la matriz de confusión
eval = function(y, pred) {
  confu = table(y, pred)
  e = 1 - sum(diag(confu)) / sum(confu)
  falsos = 1 - diag(confu) / apply(confu, 1, sum)
  error = c(e, falsos) * 100
  names(error) = c("e", "FP", "FN")
  return(list(Matriz = confu, Error = error))
}

umbrales = seq(0.1, 0.9, by = 0.1) # Se definen los umbrales desde 0.1 hasta 0.9
 # Hacemos un array debido a que las dimensiones de una matriz no permiten guardar diferentes informaci
ERRORES = array(NA, dim = c(10, 3, length(umbrales)),
                dimnames = list(paste0("Fold", 1:10),
                                c("e", "FP", "FN"),
                                umbrales))
# Se crea el bucle para probar las diferentes particiones
for (i in 1:10) {
  test  = base2[part[[i]], ]
  train = base2[-part[[i]], ]

  mod = glm(cliente ~ edad + antig_laboral + vivienda,
            family = "binomial", data = train)
  probabilidades = predict(mod, test, type = "response")
# Se crea ek bucel para los diferentes umbrales
  for (j in seq_along(umbrales)) {
    pred = factor(ifelse(probabilidades > umbrales[j], "malo", "bueno"),
                  levels = c("bueno", "malo"))
    ERRORES[i, , j] = eval(test$cliente, pred)$Error
  }
}

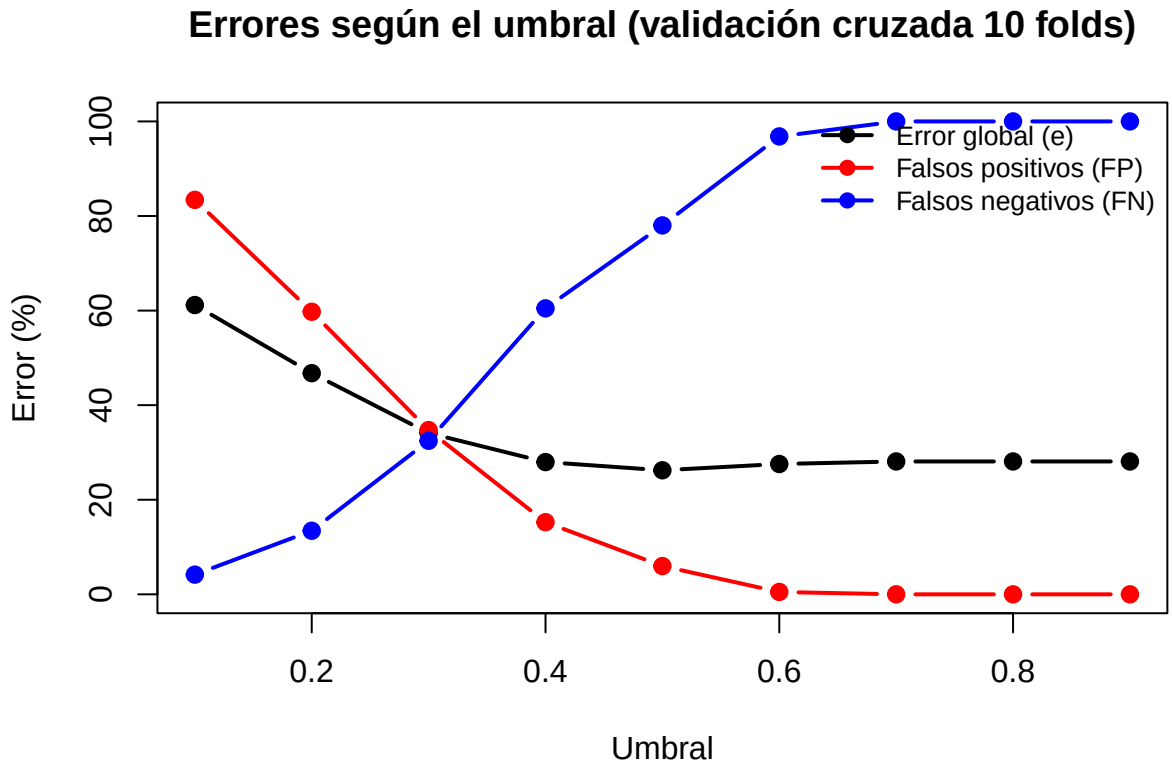
prom_errores = apply(ERRORES, c(2, 3), mean)   # matriz 3 x 9
round(prom_errores, 2)
```

```
##      0.1   0.2   0.3   0.4   0.5   0.6   0.7   0.8   0.9
```

```
## e  61.17 46.77 34.10 27.96 26.23 27.56  28.1  28.1  28.1
## FP 83.41 59.75 34.73 15.25  5.98  0.50   0.0   0.0   0.0
## FN  4.17 13.44 32.47 60.47 78.02 96.83 100.0 100.0 100.0
```

```r
matplot(umbrales, t(prom_errores), type = "b", pch = 19,
        lty = 1, lwd = 2, col = c("black", "red", "blue"),
        xlab = "Umbral", ylab = "Error (%)",
        main = "Errores según el umbral (validación cruzada 10 folds)")

legend("topright", legend = c("Error global (e)",
                              "Falsos positivos (FP)",
                              "Falsos negativos (FN)"),
       col = c("black", "red", "blue"), lty = 1, lwd = 2, pch = 19,
       bty = "n", cex = 0.85)
```



Lo que habría que comparar aquí cuál es el equilibrio y ver como cambia con la cantidad de variables que tiene cada modelo. Para así luego tener un cruterior donde se equilibrie todo.

3 K-vecinos más cercanos

A continuación se le realiza el proceso de forma manual en r y se hace una representazción gráfica antes:

3.0.1 Construcción de la matriz de distancias

Sea la base original con $n = 60$ observaciones y p columnas, donde la columna 1 es la variable respuesta y y las columnas 3 a 6 son categóricas (c):

$$\text{base2} = \begin{bmatrix} y_1 & x_{1,2} & c_{1,3} & \cdots & c_{1,6} & x_{1,7} & \cdots & x_{1,p} \\ y_2 & x_{2,2} & c_{2,3} & \cdots & c_{2,6} & x_{2,7} & \cdots & x_{2,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ y_{60} & x_{60,2} & c_{60,3} & \cdots & c_{60,6} & x_{60,7} & \cdots & x_{60,p} \end{bmatrix}_{60\times p}$$

1. Partición en entrenamiento y prueba (se eliminan las columnas categóricas 3 a 6):

$$\text{train} = \begin{bmatrix} y_1 & x_{1,2} & x_{1,7} & \cdots & x_{1,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{50} & x_{50,2} & x_{50,7} & \cdots & x_{50,p} \end{bmatrix}_{50\times(p-4)} \qquad \text{test} = \begin{bmatrix} y_{51} & x_{51,2} & x_{51,7} & \cdots & x_{51,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{60} & x_{60,2} & x_{60,7} & \cdots & x_{60,p} \end{bmatrix}_{10\times(p-4)}$$

2. Se quita la variable respuesta (`train[,-1]` y `test[,-1]`), quedando solo las $q = p - 5$ variables numéricas:

$$\mathbf{X}_{tr} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{50}^\top \end{bmatrix}_{50\times q} \qquad \mathbf{X}_{te} = \begin{bmatrix} \mathbf{x}_{51}^\top \\ \vdots \\ \mathbf{x}_{60}^\top \end{bmatrix}_{10\times q}$$

3. Se apilan por filas (`rbind`):

$$\mathbf{Z} = \begin{bmatrix} \mathbf{X}_{tr} \\ \mathbf{X}_{te} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{50}^\top \\ \mathbf{x}_{51}^\top \\ \vdots \\ \mathbf{x}_{60}^\top \end{bmatrix}_{60\times q}$$

4. Matriz de distancias euclídeas (`dist`, que por defecto usa la distancia euclídea):

$$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\| = \sqrt{\sum_{k=1}^{q} (x_{ik} - x_{jk})^2}$$

$$\mathbf{D} = \begin{bmatrix} 0 & d_{1,2} & \cdots & d_{1,50} & d_{1,51} & \cdots & d_{1,60} \\ d_{2,1} & 0 & \cdots & d_{2,50} & d_{2,51} & \cdots & d_{2,60} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ d_{50,1} & d_{50,2} & \cdots & 0 & d_{50,51} & \cdots & d_{50,60} \\ d_{51,1} & d_{51,2} & \cdots & d_{51,50} & 0 & \cdots & d_{51,60} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ d_{60,1} & d_{60,2} & \cdots & d_{60,50} & d_{60,51} & \cdots & 0 \end{bmatrix}_{60\times 60}$$

Vista por bloques:

$$\mathbf{D} = \left[\begin{array}{c|c} \mathbf{D}_{tr,tr} & \mathbf{D}_{tr,te} \\ \hline \mathbf{D}_{te,tr} & \mathbf{D}_{te,te} \end{array}\right] \qquad \begin{array}{l} \mathbf{D}_{tr,tr} : 50 \times 50 \\ \mathbf{D}_{te,tr} : 10 \times 50 \end{array}$$

$\mathbf{D}$ es simétrica ($d_{ij} = d_{ji}$) con diagonal cero. El bloque $\mathbf{D}_{te,tr}$, que corresponde a `d[51:60, 1:50]` ó tamabién puede ser `d[1:50, 51:60]` ,contiene la distancia de cada observación de prueba a cada observación de entrenamiento. Es decir cada bloque de $\mathbf{D}_{te,tr} : 10 \times 50$ es lo que necesitamos.

Para clasificar cada observación de prueba solo interesan sus distancias a las observaciones de entrenamiento, es decir, el bloque $\mathbf{D}_{te,tr}$:

$$\mathbf{D}_{te,tr} = \begin{bmatrix} d_{51,1} & d_{51,2} & \cdots & d_{51,50} \\ d_{52,1} & d_{52,2} & \cdots & d_{52,50} \\ \vdots & \vdots & \ddots & \vdots \\ d_{60,1} & d_{60,2} & \cdots & d_{60,50} \end{bmatrix}_{10\times 50}$$

Cada fila i contiene las distancias de la observación de prueba $\mathbf{x}_i$ a las 50 observaciones de entrenamiento:

$$\mathbf{d}_i = (d_{i,1},\ d_{i,2},\ \ldots,\ d_{i,50}), \qquad i = 51, \ldots, 60$$

1. Ordenar las distancias de menor a mayor. Si $d_{i,(1)} \leq d_{i,(2)} \leq \cdots \leq d_{i,(50)}$ son las distancias ordenadas, el conjunto de los k vecinos más cercanos de $\mathbf{x}_i$ es:

$$N_k(\mathbf{x}_i) = \left\{ j \in \{1, \ldots, 50\} : d_{i,j} \leq d_{i,(k)} \right\}$$

Por ejemplo, con $k = 3$, si para la observación 51 las tres menores distancias fueran

$$\mathbf{d}_{51} = \left(4.2,\ \underbrace{0.8}_{j=2},\ 3.5,\ \ldots,\ \underbrace{1.1}_{j=17},\ \ldots,\ \underbrace{0.9}_{j=40},\ \ldots \right)$$

entonces

$$N_3(\mathbf{x}_{51}) = \{2,\ 40,\ 17\}$$

2. Asignar la clase por voto mayoritario. Se toman las respuestas y_j de esos vecinos en `train` y se asigna la clase más frecuente:

$$\hat{y}_i = \arg\max_c \sum_{j \in N_k(\mathbf{x}_i)} I\left(y_j = c\right)$$

donde $I(\cdot)$ vale 1 si el vecino j pertenece a la clase c y 0 en otro caso. Por ejemplo, si $y_2 = A$, $y_{40} = A$ y $y_{17} = B$:

$$\hat{y}_{51} = A \quad (2 \text{ votos contra } 1)$$

Al repetir esto para $i = 51, \ldots, 60$ se obtiene el vector de predicciones:

$$\hat{\mathbf{y}}_{te} = (\hat{y}_{51},\ \hat{y}_{52},\ \ldots,\ \hat{y}_{60})^\top$$

Nota: Se recomienda usar un k impar para evitar evitas, por ejemplo si $k = 2$ y el resultado del proceso nos da $y_2 = A$, $y_{17} = B$, sería muy díficil saber cuál es la desición correcta.

```r
# Se obtiene las dos bases 50 obs para train, 10 para test
train=base2[1:50,-(3:6)] # si quitan las columnas de variables categóricas
test=base2[51:60,-(3:6)]
d = as.matrix(dist(rbind(train[,-1],test[,-1]))) # Se pegan las dos bases de datos obtenidos anteriorme
d2 = d[1:50,51:60]
#Luego se buscan los 3 individios con menor distancia en la primera columna
k = 3 # número de vecinos más cercanos
(ord = apply(d2,2, function(col) order(col)[1:k])) # si en lugar de order colocamos sort obtenemos los
```

```
##      51 52 53 54 55 56 57 58 59 60
## [1,] 29 47 32  2 38  5 12 25 17 28
## [2,] 44 28  4 19 10 21 11 14 22  6
## [3,] 37  6  3 48 23 40 26 50 47 47
```

Para ver que categoría de cliente tiene cada uno de esas filas, para así poder clasificarlo lo hacemos de la siguiente manera:

```r
pred = c()
for (i in 1:10) { #son 10 datos que están en el test
  tab = table(train$cliente[ord[,i]]) # se hace una tabla para obtener el conteo de cada catgoría
  pred[i] = names(tab)[which.max(tab)] # se guarda la categoría que se repite más veces
}

pred # clasificaciones de las 10 observaciones de la base test
```

```
##  [1] "malo"  "bueno" "bueno" "malo"  "malo"  "malo"  "bueno" "bueno" "bueno"
## [10] "bueno"
```

3.1 Función de k-vecinos, para diferentes tamaños de k

```r
# Función para evaluar los errores
eval = function(y, pred) {
  confu = table(y, pred)
  e = 1 - sum(diag(confu)) / sum(confu)
  falsos = 1 - diag(confu) / apply(confu, 1, sum)
  error = c(e, falsos) * 100
  names(error) = c("e", "FP", "FN")
  return(list(Matriz = confu, Error = error))
}

knn_errores = function(xtrain, ytrain, xtest, ytest, k) {
  n1 = nrow(xtrain)
  n2 = nrow(xtest)


  # Distancias de todos contra todos y nos quedamos con train vs. validación
  D  = rbind(xtrain, xtest)
  d  = as.matrix(dist(D))
  d1 = d[1:n1, (n1 + 1):(n1 + n2), drop = FALSE]

  vecinos = apply(d1, 2, function(col) order(col)[1:k])
  vecinos = matrix(vecinos, nrow = k)


  pred = c()
```

```
  for (i in 1:n2) {
    tab = table(ytrain[vecinos[, i]])
    pred[i] = names(tab)[which.max(tab)]
  }

  pred = factor(pred, levels = levels(ytrain))     # mismos niveles para la tabla 2x2
  return(eval(ytest, pred))
}
```

Ejemplo de uso de la función

```
xtrain = train[, names(train) != "cliente"] # Se quita la columna de cliente
xtest  = test[, names(test) != "cliente"] # Se quita la columna de cliente

knn_errores(xtrain, train$cliente, xtest, test$cliente, k = 3)
```

```
## $Matriz
##        pred
## y       bueno malo
##   bueno     6    3
##   malo      0    1
##
## $Error
##        e       FP       FN
## 30.00000 33.33333  0.00000
```

3.2 Validación cruzada con k-vecinos

3.2.1 Para un número k-vecinos en específico

```
cortes = createFolds(base2$cliente, k = 10) # Lista con los índices de prueba de cada fold
k = 3 # Valor fijo de k
# Matriz para guardar los errores de cada fold
ERRORES_knn = matrix(NA, nrow = 10, ncol = 3,
                     dimnames = list(paste0("Fold", 1:10),
                                     c("e", "FP", "FN")))

# Se hace un for para hacer lso cortes
for (i in 1:10) {
  test  = base2[cortes[[i]], -(3:6)]  # Se quitan las variables categóricas
  train = base2[-cortes[[i]], -(3:6)]

  # Matrices de predictores sin la variable respuesta
  xtrain = train[, names(train) != "cliente"]
  xtest  = test[, names(test) != "cliente"]

  ERRORES_knn[i, ] = knn_errores(xtrain, train$cliente,
                                 xtest, test$cliente, k = k)$Error
}

round(ERRORES_knn, 2)           # Errores por corte
```

```
##            e    FP    FN
## Fold1  30.41 13.79 72.80
## Fold2  28.76 14.69 64.80
```

```
## Fold3  31.01 15.31 71.20
## Fold4  32.88 16.61 74.40
## Fold5  34.16 20.63 68.80
## Fold6  28.99 15.31 64.00
## Fold7  29.89 16.87 63.20
## Fold8  31.15 14.73 73.39
## Fold9  31.31 15.67 71.20
## Fold10 31.46 16.87 68.80
```

```
round(colMeans(ERRORES_knn), 2) # Errores en general
```

```
##     e    FP    FN
## 31.00 16.05 69.26
```

3.2.2 Para un diferentes números de k-vecinos

```
# Validación cruzada con k-NN para una distancia dada, se le puede cambiar el tipo de distancia
cv_knn = function(cortes, ks, metodo = "euclidean") {

  # Matriz para acumular los errores de cada k (inicia en 0 para ir sumando)
  suma = matrix(0, nrow = length(ks), ncol = 3,
                dimnames = list(paste0("k=", ks), c("e", "FP", "FN")))

  for (i in seq_along(cortes)) {                    # Recorre cada fold
    test   = base2[cortes[[i]], -(3:6)]             # Fold i como prueba, sin categóricas
    train  = base2[-cortes[[i]], -(3:6)]            # Resto como entrenamiento
    xtrain = train[, names(train) != "cliente"]     # Predictores sin la respuesta
    xtest  = test[, names(test) != "cliente"]
    n1 = nrow(xtrain)                               # Cantidad de observaciones de entrenamiento

    # Distancias entre todos, y se dejan solo filas = train, columnas = test
    d1  = as.matrix(dist(rbind(xtrain, xtest), method = metodo))[1:n1, -(1:n1), drop = FALSE]
    ord = apply(d1, 2, order)                       # Por columna, índices de train del más cercano al má

    for (j in seq_along(ks)) {                      # Recorre cada valor de k
      vecinos = ord[1:ks[j], , drop = FALSE]        # Los k vecinos más cercanos de cada obs. de prueba

      # Voto mayoritario: clase más frecuente entre los vecinos
      pred = apply(vecinos, 2, function(v) {
        tab = table(train$cliente[v])               # Cuenta las clases de los vecinos
        names(tab)[which.max(tab)]                  # Clase con más votos
      })

      pred = factor(pred, levels = levels(train$cliente))     # Mismos niveles que la respuesta real
      suma[j, ] = suma[j, ] + eval(test$cliente, pred)$Error  # Acumula e, FP y FN
    }
  }
  return(suma / length(cortes))                     # Promedio de los errores entre folds
}
```

Ejemplo de ejecución

```
cortes = createFolds(base2$cliente, k = 10)
ks = seq(1,15, by = 2)
valid_cruz = cv_knn(cortes,ks,metodo = "euclidean")
```

```
valid_cruz
```

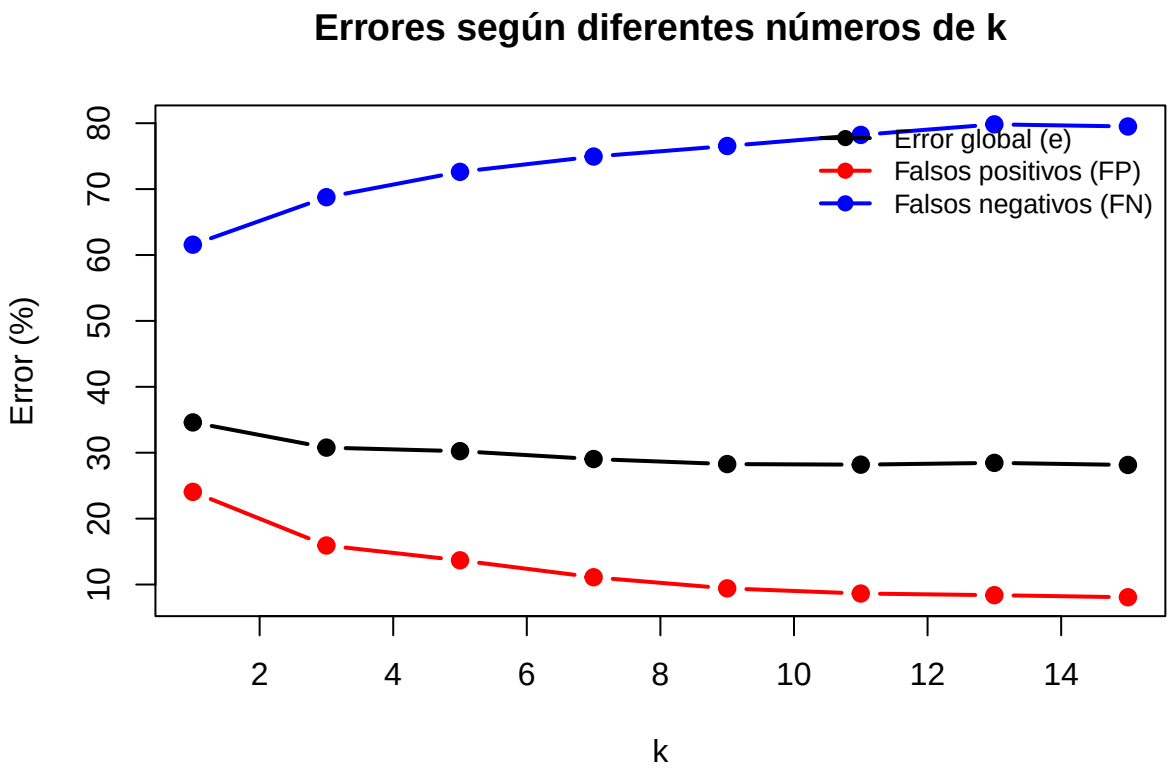
```
##             e        FP       FN
## k=1  34.59974 24.060933 61.56710
## k=3  30.77485 15.926038 68.77032
## k=5  30.23614 13.674373 72.61613
## k=7  29.04295 11.106975 74.93613
## k=9  28.27804  9.417320 76.53871
## k=11 28.18866  8.635874 78.22065
## k=13 28.45919  8.385776 79.82387
## k=15 28.14387  8.072884 79.50258
```

Validación cruzada: calibración de k-NN

Concepto	Descripción	En k-NN
Calibración	Ajuste de los parámetros que se pueden cambiar según el método, buscando el que dé menor error en la validación cruzada.	Se calibran k y la distancia.
k	Número de vecinos más cercanos que votan para asignar la clase de una observación de prueba.	Se prueban varios valores (por ejemplo, $k = 1, 3, 5, \ldots, 15$) y se elige el de menor error.
Distancia	Forma de medir qué tan cerca están dos observaciones.	Euclídea: $d_{ij} = \sqrt{\sum_k (x_{ik} - x_{jk})^2}$. Manhattan: $d_{ij} = \sum_k \lvert x_{ik} - x_{jk} \rvert$.
Para distancia euclídea	Una vez fijada la distancia, solo queda calibrar el número de vecinos.	Se calibra únicamente k.

```
matplot(ks, valid_cruz, type = "b", pch = 19,
        lty = 1, lwd = 2, col = c("black", "red", "blue"),
        xlab = "k", ylab = "Error (%)",
        main = "Errores según diferentes números de k")

legend("topright", legend = c("Error global (e)",
                              "Falsos positivos (FP)",
                              "Falsos negativos (FN)"),
       col = c("black", "red", "blue"), lty = 1, lwd = 2, pch = 19,
       bty = "n", cex = 0.85)
```



Se puede ver que conforme el número de k-vecinos aumenta, la cantidad de FN aumenta, por lo que se quiere buscar es un equilibrio entre los tres tipos de error y en el gráfico anterior para k=1 es cuándo los tres errores están menor distanciados.

3.2.3 Comparación de dos métodos de distancia una cierta cantidad de veces

Esto lo debemos hacer cuándo los resultados no son muy consistentes, hay que iterarlo en distintas ocasiones y luego promediar esos resultados, para observar hacia a donde convergen.

```
reps = 2

# Matriz con los errores de cada repetición, para la distancia euclidea
k1_euc = matrix(NA, nrow = reps, ncol = 3,
                dimnames = list(paste0("Rep", 1:reps), c("e", "FP", "FN")))
k1_man = k1_euc # Matriz de errores distancia de Manhattan
# for para hacer los diferentes iteraciones, en este caso son 2
for (r in 1:reps) {
  cortes = createFolds(base2$cliente, k = 10) # Cortes distintos en cada repetición
  k1_euc[r, ] = cv_knn(cortes, ks = 1, metodo = "euclidean")
  k1_man[r, ] = cv_knn(cortes, ks = 1, metodo = "manhattan")
}

# Promedio de las 3 repeticiones para cada método
comparacion_k1 = rbind(Euclidea  = colMeans(k1_euc),
                       Manhattan = colMeans(k1_man))
round(comparacion_k1, 2)
```

```
##                e    FP    FN
## Euclidea   34.84 24.09 62.33
## Manhattan 34.48 23.72 62.01
```

```
cortes   = createFolds(base2$cliente, k = 10)   # Índices de prueba de cada fold
umbrales = seq(0.1, 0.9, by = 0.1)              # Puntos de corte a calibrar

# Matriz para acumular los errores de cada umbral (filas = umbrales)
suma = matrix(0, nrow = length(umbrales), ncol = 3,
              dimnames = list(umbrales, c("e", "FP", "FN")))

for (i in seq_along(cortes)) {                  # Recorre cada fold
  test  = base2[cortes[[i]], ]                  # Fold i como prueba
  train = base2[-cortes[[i]], ]                 # Resto como entrenamiento

  # Modelo ajustado una sola vez por fold
   mod = glm(cliente ~ ., family = "binomial", data = train)  # Todas las variables como predictoras
  probabilidades = predict(mod, test, type = "response")  # P(malo) de cada obs. de prueba

  for (j in seq_along(umbrales)) {              # Recorre cada umbral
    pred = factor(ifelse(probabilidades > umbrales[j], "malo", "bueno"),
                  levels = c("bueno", "malo"))  # Clasifica según el umbral
    suma[j, ] = suma[j, ] + eval(test$cliente, pred)$Error  # Acumula e, FP y FN
  }
}

prom_errores_log = suma / length(cortes)        # Promedio de los 10 folds
round(prom_errores_log, 2)
```

```
##         e    FP    FN
## 0.1 49.69 66.96  5.53
## 0.2 33.92 41.33 14.97
## 0.3 25.38 24.00 28.91
## 0.4 22.79 14.83 43.16
## 0.5 22.25  8.35 57.81
## 0.6 23.15  4.69 70.38
## 0.7 24.72  2.28 82.15
## 0.8 26.64  1.13 91.92
## 0.9 27.74  0.09 98.48
```

```
umbrales[which.min(prom_errores_log[, "e"])]    # Umbral con menor error global
```

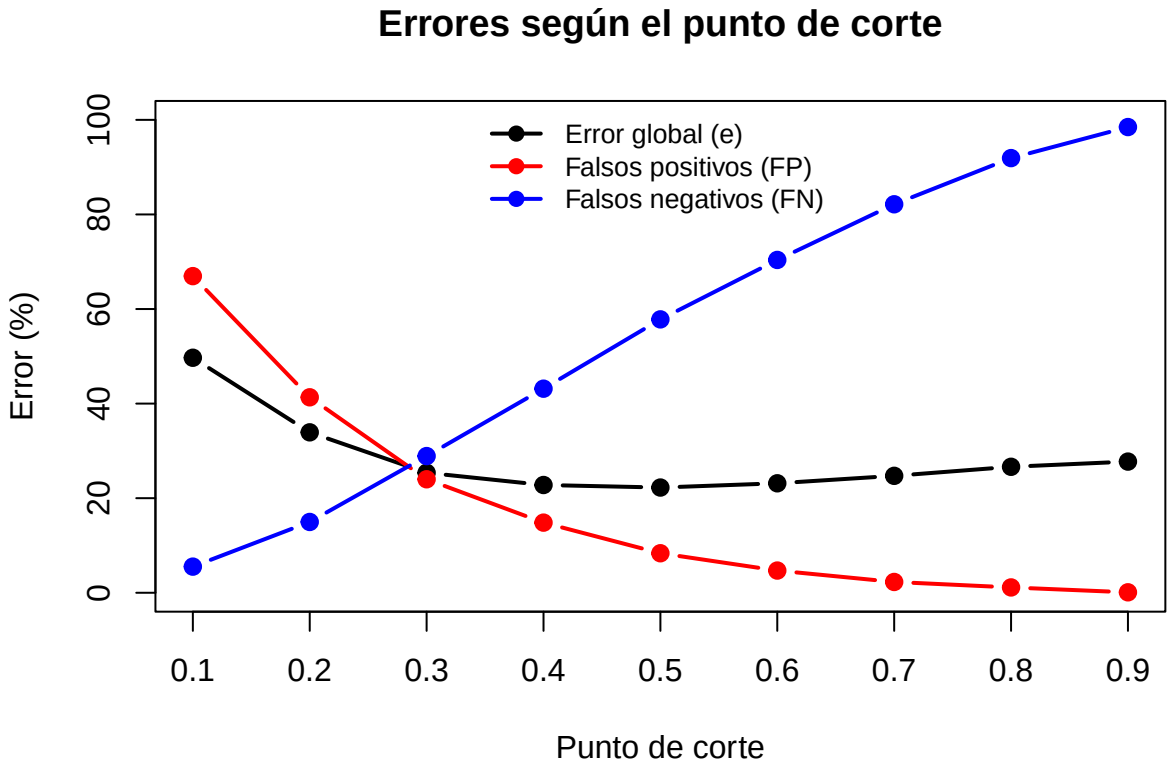
```
## [1] 0.5
```

```
matplot(umbrales, prom_errores_log, type = "b", pch = 19,
        lty = 1, lwd = 2, col = c("black", "red", "blue"),
        ylim = c(0, 100), xaxt = "n",
        xlab = "Punto de corte", ylab = "Error (%)",
        main = "Errores según el punto de corte")

axis(1, at = umbrales)                          # Eje X con los umbrales probados

legend("top", legend = c("Error global (e)",
                         "Falsos positivos (FP)",
                         "Falsos negativos (FN)"),
       col = c("black", "red", "blue"), lty = 1, lwd = 2, pch = 19,
```

```
       bty = "n", cex = 0.85)
```




```
cortes = createFolds(base2$cliente, k = 10)     # Índices de prueba de cada fold
umbral = 0.5                                    # Punto de corte fijo (o el mejor de la calibración ante

vars    = names(base2)[names(base2) != "cliente"]    # Todas las variables predictoras
modelos = c("Completo", paste("Sin", vars))     # Nombres: modelo completo y uno sin cada variable

# Matriz para acumular los errores de cada modelo
suma = matrix(0, nrow = length(modelos), ncol = 3,
              dimnames = list(modelos, c("e", "FP", "FN")))

for (i in seq_along(cortes)) {                  # Recorre cada fold
  test  = base2[cortes[[i]], ]                  # Fold i como prueba
  train = base2[-cortes[[i]], ]                 # Resto como entrenamiento

  for (j in seq_along(modelos)) {               # Recorre cada modelo
    if (j == 1) {
      predictoras = vars                        # Modelo completo
    } else {
      predictoras = vars[-(j - 1)]              # Se quita la variable j - 1
    }
    formula = reformulate(predictoras, response = "cliente")  # Arma cliente ~ x1 + x2 + ...

    mod  = glm(formula, family = "binomial", data = train)
    prob = predict(mod, test, type = "response")              # P(malo)
```

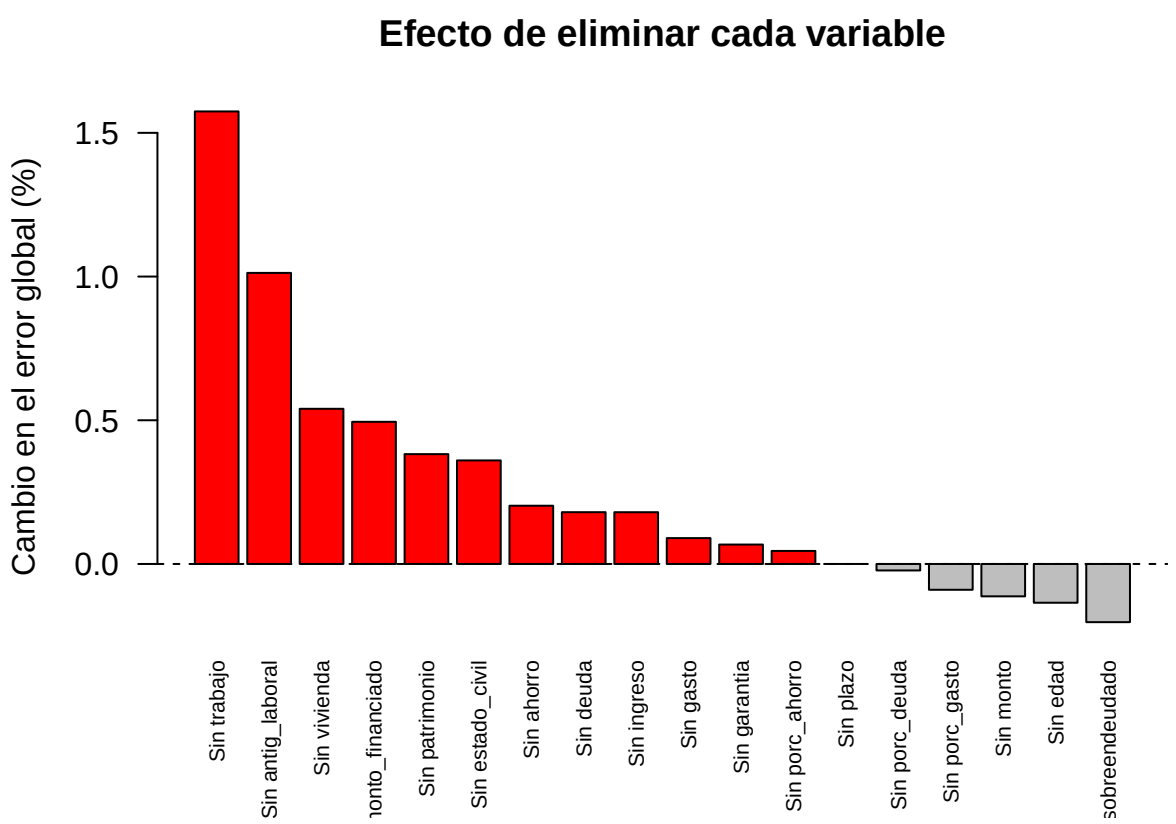
```
    pred = factor(ifelse(prob > umbral, "malo", "bueno"),
                  levels = c("bueno", "malo"))                # Clasifica con el umbral fijo
    suma[j, ] = suma[j, ] + eval(test$cliente, pred)$Error    # Acumula e, FP y FN
  }
}

prom_errores_var = suma / length(cortes)       # Promedio de los 10 folds
round(prom_errores_var, 2)
```

```
##                          e   FP    FN
## Completo             22.09 8.36 57.25
## Sin edad             21.96 8.39 56.69
## Sin antig_laboral    23.11 7.95 61.89
## Sin vivienda         22.63 8.04 59.97
## Sin estado_civil     22.45 8.10 59.17
## Sin trabajo          23.67 8.64 62.13
## Sin ingreso          22.27 8.73 56.93
## Sin gasto            22.18 8.51 57.17
## Sin deuda            22.27 8.64 57.17
## Sin ahorro           22.30 8.73 57.01
## Sin patrimonio       22.47 8.67 57.81
## Sin porc_deuda       22.07 8.48 56.85
## Sin porc_ahorro      22.14 8.45 57.17
## Sin porc_gasto       22.00 8.36 56.93
## Sin sobreendeudado   21.89 8.29 56.69
## Sin plazo            22.09 8.39 57.17
## Sin monto            21.98 8.39 56.77
## Sin garantia         22.16 8.45 57.25
## Sin monto_financiado 22.59 8.86 57.73
```

```
dif_e = prom_errores_var[-1, "e"] - prom_errores_var["Completo", "e"]  # Cambio en el error al quitar c

barplot(sort(dif_e, decreasing = TRUE), las = 2, cex.names = 0.7,
        col = ifelse(sort(dif_e, decreasing = TRUE) > 0, "red", "gray"),
        ylab = "Cambio en el error global (%)",
        main = "Efecto de eliminar cada variable")
abline(h = 0, lty = 2)                         # Línea de referencia: sin cambio
```



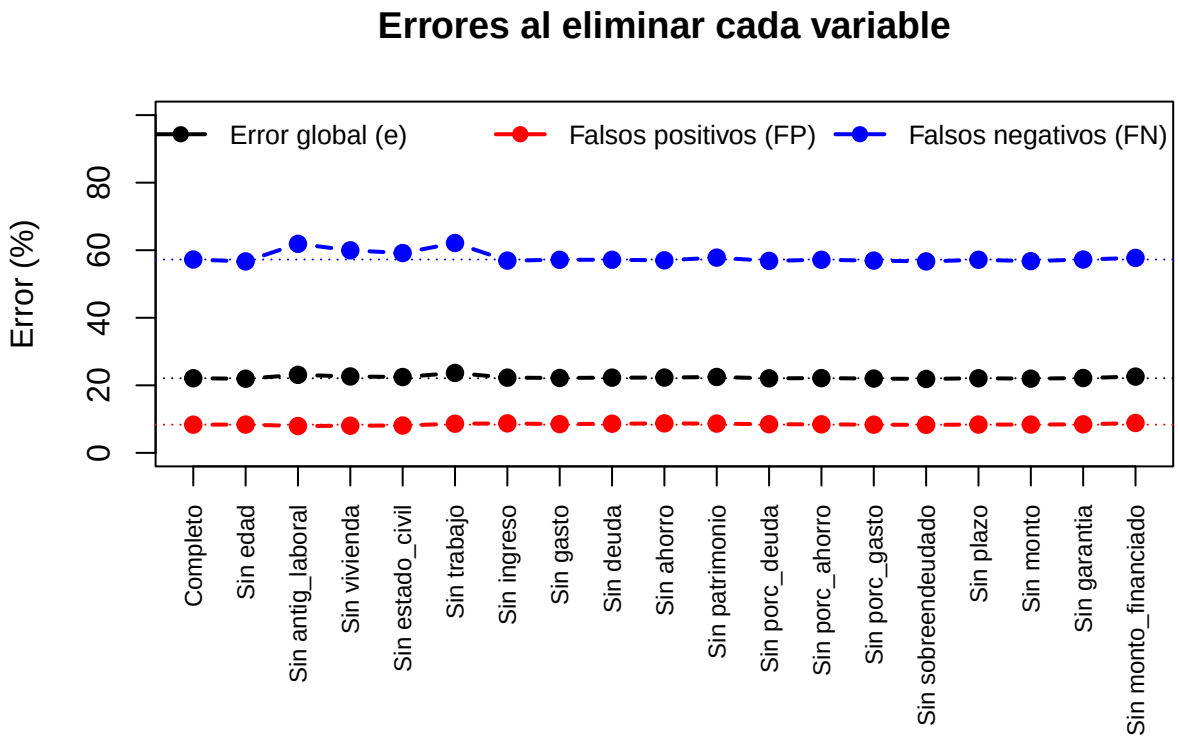

```r
par(mar = c(9, 4, 4, 2))                          # Más margen abajo para los nombres de las variables

matplot(1:nrow(prom_errores_var), prom_errores_var, type = "b", pch = 19,
        lty = 1, lwd = 2, col = c("black", "red", "blue"),
        ylim = c(0, 100), xaxt = "n",             # Se quita el eje X automático
        xlab = "", ylab = "Error (%)",
        main = "Errores al eliminar cada variable")

axis(1, at = 1:nrow(prom_errores_var),            # Una marca por modelo
     labels = rownames(prom_errores_var),         # Nombres: Completo, Sin edad, ...
     las = 2, cex.axis = 0.75)                    # Nombres verticales y más pequeños

abline(h = prom_errores_var["Completo", ],        # Referencia: errores del modelo completo
       col = c("black", "red", "blue"), lty = 3)

legend("top", legend = c("Error global (e)",
                         "Falsos positivos (FP)",
                         "Falsos negativos (FN)"),
       col = c("black", "red", "blue"), lty = 1, lwd = 2, pch = 19,
       bty = "n", cex = 0.85, horiz = TRUE)       # Leyenda en una sola fila
```



4 Métodos basados en árboles

Para estos métodos son necesarias las siguientes librerías:

```
library(rpart)
library(rattle)
```

```
## Warning: package 'rattle' was built under R version 4.5.3
```

```
library(DT)
```

```
## Warning: package 'DT' was built under R version 4.5.3
```

```
library(adabag)
```

```
## Warning: package 'adabag' was built under R version 4.5.3
## Warning: package 'foreach' was built under R version 4.5.3
## Warning: package 'doParallel' was built under R version 4.5.3
## Warning: package 'iterators' was built under R version 4.5.3
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.5.3
```

1. Primero se divide la base de datos en 10 particiones

```
library(caret)
part = createFolds(1:nrow(base2), k = 10)
```

2. Dividimos la base de datos en 90% de entrenamiento y 10% trest

```
test = base2[part[[1]],] #obtiene las filas de la primera partición
train = base2[-part[[1]],]
```

3. Estimamos el modelo con los datos de entrenamiento

```
mod1 = rpart(cliente ~ ., method="class", data=train)
```

4. Usando la función `eval()` que creamos anteriormente, observamos los diferentes errores de nuestra estimación:

```
eval = function(y, pred) {
  confu = table(y, pred)
  e = 1 - sum(diag(confu)) / sum(confu)
  falsos = 1 - diag(confu) / apply(confu, 1, sum)
  error = c(e, falsos) * 100
  names(error) = c("e", "FP", "FN")
  return(list(Matriz = confu, Error = error))
}
```

```
pred=predict(mod1,newdata=test,type="class")
eval(test$cliente,pred)
```

```
## $Matriz
##        pred
## y       bueno malo
##   bueno   289   31
##   malo     76   49
##
## $Error
##        e       FP       FN
## 24.04494  9.68750 60.80000
```

4.1 Validación cruzada, para el método de árboles

Para cada una de las particiones que hicimos con la función `createFolds()`, evaluamos sus errores y se promedian a lo último

```
ERRORES <- matrix(NA, nrow = 10, ncol = 3,
                  dimnames = list(paste0("Fold", 1:10), c("e", "FP", "FN")))

for (i in 1:10) {
  test_i  <- base2[part[[i]], ]
  train_i <- base2[-part[[i]], ]

  mod  <- rpart(cliente ~ ., method = "class", data = train_i)
  pred <- predict(mod, test_i, type = "class")

  res <- eval(test_i$cliente, pred)
  ERRORES[i, ] <- c(res$Error, res$FP, res$FN)
}

ERRORES
```

```
##               e        FP       FN
## Fold1  24.04494  9.687500 60.80000
## Fold2  28.02691  5.211726 78.41727
## Fold3  23.19820 10.778443 60.90909
## Fold4  22.86996  5.806452 61.76471
## Fold5  25.67568  9.148265 66.92913
## Fold6  25.00000  9.375000 65.32258
## Fold7  21.39640  5.214724 66.10169
## Fold8  24.09910  9.287926 63.63636
## Fold9  25.00000 10.828025 59.23077
## Fold10 26.80180 12.923077 64.70588
```

```
colMeans(ERRORES)
```

```
##         e        FP        FN
## 24.611298  8.826114 64.781749
```

El proceso anterior tiene un detalle, que la función `createFolds()` hace las particiones de manera aleatoria, de modo que para que los resultados sean consistentes se necesita realizar el proceso ciertas cantidad de veces para poder asegurar que el resultado converge hacia una dirección.

```
library(rpart)
library(caret)

R <- 14 # número de repeticiones
K <- 10   # número de folds

ERRORES_REP <- matrix(NA, nrow = R, ncol = 3,
                      dimnames = list(paste0("Rep", 1:R), c("e", "FP", "FN")))

for (r in 1:R) {
  part <- createFolds(base2$cliente, k = K)   # folds nuevos en cada repetición

  ERRORES <- matrix(NA, nrow = K, ncol = 3,
                    dimnames = list(paste0("Fold", 1:K), c("e", "FP", "FN")))

  for (i in 1:K) {
    test_i  <- base2[part[[i]], ]
    train_i <- base2[-part[[i]], ]

    mod  <- rpart(cliente ~ ., method = "class", data = train_i)
    pred <- predict(mod, test_i, type = "class")

    res <- eval(test_i$cliente, pred)
    ERRORES[i, ] <- c(res$Error, res$FP, res$FN)
  }

  ERRORES_REP[r, ] <- colMeans(ERRORES)   # promedio de los K folds en esta repetición
}

ERRORES_REP
colMeans(ERRORES_REP)        # error promedio final
```

4.2 Gráfico de árboles

```
# Puede usar x11(), para visualizar de mejor el árbol
x11()
fancyRpartPlot(mod1 , palettes=c("Greens", "Reds"))
```

Para poder obtener la misma proporción que está en el nodo, obtengalo de la siguiente manera:

```
prop.table(table(train$cliente))
```

```
##
## bueno  malo
## 0.719 0.281
```

Si yo quiero dirigirme hacia la derecha de la figura la condición lógica, debe ser planteda al revés de modo que si vemos la primero condición es si la antigüedad laboral es mayor o igual a 2.5, por eso en el siguiente código la condición se plante al revés.

```
prop.table(table(train$antig_laboral < 2.5))
```

```
##
##   FALSE    TRUE
## 0.66425 0.33575
```

Este método para las variables númericas realiza una partición en un cierto valor para dividrir y de esa manera realiza la categorízación. El caso de las variables ordinales y categóricas este método realiza un filtrado para ir haciendo los distintos caminos.